

REST 練習 9 (期末整合) : Flight Booking CRUD REST API

Lecture

這是本課程最後一題，目標不是學新概念，而是把 **rs01~rs08** 教過的每一塊拼成一個完整的、像樣的 **REST Service**：JSON 序列化 (rs03)、路徑參數定位單筆資源 (rs04)、Query Parameter 篩選集合 (rs05)、POST 建立資源與外鍵驗證 (rs06)、PUT/DELETE 與統一錯誤格式 (rs07)、Service/Resource 分層與可測試性 (rs08)——這次全部套用在一個新的資料模型上：**訂位 (SBOOK)**，而不是繼續用 rs01~rs08 一路用到的航班 (SFLIGHT)。

選訂位當期末整合的題材是刻意的：**SBOOK** 的主鍵是 **CARRID + CONNID + FLDATE + BOOKID** 四個欄位 (比 **SFLIGHT** 的三個鍵多一個 **BOOKID**)，而且 **SBOOK** 本身還有兩個外鍵要檢查——訂位一定要掛在一個真實存在的航班 (**SFLIGHT**) 底下，也要對應一個真實存在的客戶 (**SCUSTOM**)。這讓「建立訂位」這個 **POST** 動作比 rs06「建立航班」複雜一截：rs06 的 **POST /flights** 只驗證 **carriid** 一個外鍵，這題的 **POST /bookings** 要驗證航班存在、客戶存在、**class** 艙等代碼合法，三種不同的業務規則、三種不同的錯誤代碼，全部要在同一個 **Service** 方法裡按順序檢查。

另一個這題才出現的細節是 **BOOKID** 由伺服器自動編號：呼叫端建立訂位時不會 (也不該) 自己指定 **bookid**，而是由伺服器查詢「這個航班目前訂位到第幾號」再自動 +1——這是很多真實系統「新增一筆明細」時常見的模式 (發票號、訂單號、序號都是類似邏輯)，這題用 **SELECT MAX(bookid) ... + 1** 示範最簡單的做法，也留了一個思考題討論這個做法在高並發情境下的風險。

架構上完全沿用 rs08 建立的分層：**ZCL_RS09_BOOKING_SERVICE** 是純 **ABAP** 商業邏輯層 (不依賴任何 **IF_REST_*** 型別)，兩個 **Resource** 類別 (**ZCL_RS09_BOOKINGS** 集合、**ZCL_RS09_BOOKING** 單筆) 都只做「解析 request → 呼叫 Service → 組 response」的薄層轉接。這題新增的驗證邏輯 (**validate_booking_fields** 檢查 **class** 是否為合法艙等代碼) 一樣寫進 **Service**，一樣有 **ABAP Unit** 測試覆蓋——這證明 **rs08** 立下的分層方式不是只為了那一題量身打造的示範，而是一套可以套用到任何新資料模型的通用做法。

學習目標

- 綜合運用 rs03~rs08 教過的每一項技術，設計一個全新資料模型 (**SBOOK** 訂位) 的完整 **CRUD REST API**：集合 **GET** (含篩選)、單筆 **GET**、**POST** 建立、**PUT** 更新、**DELETE** 刪除
- 學會「伺服器自動編號」的設計模式：**BOOKID** 不由呼叫端指定，而是伺服器查詢目前最大值 +1 自動產生——這跟 rs06 **POST /flights** 呼叫端自己給定完整主鍵 (**carriid/connid/fldate**) 的情境不同，體會兩種「誰決定新資源身分」的設計差異
- 學會一個 **POST** 動作可以疊加多種外鍵/業務規則驗證 (航班存在、客戶存在、艙等代碼合法)，並且知道要先檢查外鍵、再檢查業務規則 (因為外鍵不存在時，繼續檢查其他欄位沒有意義)
- 驗證 rs08 建立的「Service 不依賴 REST 框架、Resource 只做轉接」分層方式可以套用到全新的資料模型，不是量身訂做的一次性技巧
- 體會「集合資源」跟「單筆資源」即使操作的是同一份資料，也需要各自獨立的 **Resource** 類別 (**ZCL_RS09_BOOKINGS** vs **ZCL_RS09_BOOKING**)，呼應 rs04 一開始教的集合/單筆分工原則

事前準備

ADT 端物件已由課程準備好 (\$TMP) :

- ZCX_RS09_BOOKING_ERROR——業務例外，跟 rs07/rs08 同設計
- ZCL_RS09_BOOKING_SERVICE——純 ABAP 商業邏輯層，公開 `ty_keys/ty_booking/ty_create/ty_update` 型別，附 ABAP Unit 測試 (8 個測試方法，涵蓋 `parse_keys` 與 `validate_booking_fields`)
- ZCL_RS09_APP——Application Class，router 掛兩條路徑：`/bookings` (集合) → `ZCL_RS09_BOOKINGS`、`/bookings/{carrid}/{connid}/{fldate}/{bookid}` (單筆) → `ZCL_RS09_BOOKING`；覆寫 `HANDLE_CSRF_TOKEN` 跳過檢查
- ZCL_RS09_BOOKINGS——訂位集合：`GET` (`carrid/customid` 篩選)、`POST` (建立訂位)
- ZCL_RS09_BOOKING——單筆訂位：`GET/PUT/DELETE`

題目需求 (對照已建好的答案物件)

1. ZCL_RS09_BOOKING_SERVICE 的型別與方法總覽：

```
``abap TYPES: BEGIN OF ty_keys, carrid TYPE s_carr_id, connid TYPE s_conn_id, fldate TYPE s_date, bookid TYPE s_book_id, END OF ty_keys, BEGIN OF ty_booking, carrid TYPE s_carr_id, connid TYPE s_conn_id, fldate TYPE s_date, bookid TYPE s_book_id, customid TYPE s_customer, class TYPE s_class, passname TYPE s_passname, forcuram TYPE s_f_cur_pr, forcurkey TYPE s_curr, END OF ty_booking, tt_booking TYPE STANDARD TABLE OF ty_booking WITH EMPTY KEY, BEGIN OF ty_create, "POST /bookings 的 body: 主鍵三碼 + 業務欄位 ( bookid 由伺服器決定 ) carrid TYPE s_carr_id, connid TYPE s_conn_id, fldate TYPE s_date, customid TYPE s_customer, class TYPE s_class, passname TYPE s_passname, forcuram TYPE s_f_cur_pr, forcurkey TYPE s_curr, END OF ty_create, BEGIN OF ty_update, "PUT 的 body: 只有業務欄位，鍵值已經在 URL 裡 customid TYPE s_customer, class TYPE s_class, passname TYPE s_passname, forcuram TYPE s_f_cur_pr, forcurkey TYPE s_curr, END OF ty_update. ``
```

ty_create 跟 **ty_update** 除了 **carrid/connid/fldate** 之外欄位完全一樣——這不是巧合，這題的 **PUT** 語意是「用 body 內容整筆換掉訂位的業務欄位」(**customid/class/passname/forcuram/forcurkey** 全部一起換)，呼應 rs07 思考題 3 提過的「**PUT** 應該是整筆覆蓋，不是只改一兩個欄位」——這題的 **update_booking** 確實會重新驗證 **customid** (呼叫 **check_customer_exists**)，示範一個更貼近標準 **PUT** 語意的更新設計。

1. **create_booking** (建立訂位，展示多重驗證疊加 + 伺服器自動編號)：

```
``abap METHOD create_booking. check_flight_exists( iv_carrid = is_create-carrid iv_connid = is_create-connid iv_fldate = is_create-fldate ).
```

```
check_customer_exists( is_create-customid ).
```

```
validate_booking_fields( iv_customid = is_create-customid iv_class = is_create-class iv_passname = is_create-passname ).
```

```
SELECT MAX( bookid ) FROM sbook WHERE carrid = @is_create-carrid AND connid = @is_create-connid AND fldate = @is_create-fldate INTO @DATA(lv_max_bookid).
```

```
DATA(lv_next_bookid) = lv_max_bookid + 1.
```

```
DATA ls_sbook TYPE sbook. ls_sbook-carrid = is_create-carrid. ls_sbook-connid = is_create-connid. ls_sbook-fldate = is_create-fldate. ls_sbook-bookid = lv_next_bookid. ls_sbook-customid = is_create-customid.
```

ls_sbook-class = is_create-class. ls_sbook-passname = is_create-passname. ls_sbook-forcuram = is_create-forcuram. ls_sbook-forcurkey = is_create-forcurkey. ls_sbook-order_date = sy-datum.

INSERT sbook FROM ls_sbook.

```
rs_booking = get_booking( VALUE ty_keys( carrid = is_create-carrid connid = is_create-connid fldate = is_create-fldate bookid = lv_next_bookid ) ). ENDMETHOD. ``
```

驗證順序是刻意的：先查外鍵（航班、客戶存在與否），最後才查業務規則（**class** 合不合法）——如果 **carrid** 打錯，根本不用管 **class** 填得對不對，先讓呼叫端知道「你指定的航班不存在」比較有意義。

1. **ZCL_RS09_BOOKINGS~GET**（集合篩選，沿用 rs05 的布林旗標 OR 動態組合 WHERE）：

```
``abap DATA(lv_has_carrid) = mo_request->has_uri_query_parameter( 'carrid' ). DATA(lv_has_customid) = mo_request->has_uri_query_parameter( 'customid' ).
```

```
DATA(lv_carrid) = COND s_carr_id( WHEN lv_has_carrid = abap_true THEN mo_request->get_uri_query_parameter( 'carrid' ) ELSE ` ` ). DATA(lv_customid) = COND s_customer( WHEN lv_has_customid = abap_true THEN mo_request->get_uri_query_parameter( 'customid' ) ELSE ` ` ).
```

```
DATA(lt_booking) = zcl_rs09_booking_service=>list_bookings( iv_carrid = lv_carrid iv_has_carrid = lv_has_carrid iv_customid = lv_customid iv_has_customid = lv_has_customid ). ``
```

對應的 Service 方法：

```
``abap METHOD list_bookings. SELECT carrid, connid, fldate, bookid, customid, class, passname, forcuram, forcurkey FROM sbook WHERE ( carrid = @iv_carrid OR @iv_has_carrid = @abap_false ) AND ( customid = @iv_customid OR @iv_has_customid = @abap_false ) ORDER BY carrid, connid, fldate, bookid INTO TABLE @rt_booking UP TO 50 ROWS. ENDMETHOD. ``
```

1. **ZCL_RS09_BOOKINGS~POST**（建立訂位，201 + Location header，沿用 rs06 的手法）：

```
``abap DATA(ls_booking) = zcl_rs09_booking_service=>create_booking( ls_create ).
```

```
DATA(lv_location) = |/bookings/{ ls_booking-carrid }/{ ls_booking-connid }/{ CONV string( ls_booking-fldate ) }/{ ls_booking-bookid }|.
```

```
mo_response->set_header_field( iv_name = 'Location' iv_value = lv_location ). mo_response->set_status( cl_rest_status_code=>gc_success_created ). ``
```

fldate 一樣要用 **CONV string(...)** 才能保留不帶短橫線的 8 碼格式（rs06/rs07/rs08 都教過的地雷，這題第四次出現，這次沒有再犯）。

1. **ZCL_RS09_BOOKING**（單筆 GET/PUT/DELETE）完全比照 rs08 的薄層寫法，多一個私有方法 **parse_request_keys()** 把「從 **mo_request** 讀四個路徑參數、呼叫 **zcl_rs09_booking_service=>parse_keys**」封裝起來，三個方法（GET/PUT/DELETE）都先呼叫它取得 **ls_keys**，不重複寫四行 **get_uri_attribute**。完整程式碼見 **zcl_rs09_booking.clas.abap**。

SICF 掛載步驟

沿用既有的 `/sap/bc/zrest_training` 分類節點，這題新增子節點：

1. SICF 交易碼，在 `zrest_training` node 上右鍵 → **New Sub-Element**，Service Name 填 `rs09`
2. Handler List 掛 `ZCL_RS09_APP`
3. Activate Service
4. 用 `SPROX_HTTP_REQUEST` 測試，**URL 一律填完整網址**（`http://<主機>:<port>/sap/bc/zrest_training/rs09/...?sap-client=130`）：
 - 建立訂位（**201**）：`POST .../bookings`，Req. Body（挑一個已知存在的航班，例如 `rs07/rs08` 測過的 `LH/400/20190104`，客戶 id 先用 `SE16` 查一筆 `SCUSTOM` 真實存在的 ID）：

``json

```
{"carrid":"LH","connid":400,"fldate":"20190104","customid":"00000001","class":"Y","passname":"Mickey Mouse","forcuram":250.00,"forcurkey":"EUR"}`
```

確認 Resp. Header 有 Location: `/bookings/LH/0400/20190104/<自動產生的bookid>`

- 缺必填欄位（**400 VALIDATION_FAILED**）：拿掉 `passname` 再送一次
- 不合法的艙等（**400 INVALID_CLASS**）：`class` 給 `"X"`
- 航班不存在（**400 INVALID_FLIGHT**）：`carrid` 給 `"ZZ"`
- 客戶不存在（**400 INVALID_CUSTOMER**）：`customid` 給一個資料庫沒有的號碼，例如 `"99999999"`
- 查詢單筆（**GET**）：用剛剛 Location header 給的網址（去掉開頭 `/`、補上主機與 `rs09` 路徑）
- 更新訂位（**PUT**，**200**）：同一個網址，Req. Body 換一個艙等/乘客名
- 刪除訂位（**DELETE**，**204**）：同一個網址，Method 選 DELETE（若 `SPROX_HTTP_REQUEST` 沒有 DELETE 選項，這步驟跟 `rs07/rs08` 一樣待補測）
- 查無訂位（**404 BOOKING_NOT_FOUND**）：刪除後再 GET 同一個網址
- 集合篩選：`GET .../bookings?carrid=LH`、`GET .../bookings?customid=00000001`

預期輸出（範例）

建立成功：**201**，Location: `/bookings/LH/0400/20190104/<bookid>`

```
{"carrid":"LH","connid":400,"fldate":"2019-01-04","bookid":<bookid>,"customid":"1","class":"Y","passname":"Mickey Mouse","forcuram":250.00,"forcurkey":"EUR"}
```

各種驗證失敗：

```
{"errorCode":"VALIDATION_FAILED","message":"customid、class、passname 都必填"}
{"errorCode":"INVALID_CLASS","message":"class 必須是 F (頭等艙)、C (商務艙) 或 Y (經濟艙) 之一"}
{"errorCode":"INVALID_FLIGHT","message":"找不到航班 ZZ/0400/20190104，無法建立訂位"}
{"errorCode":"INVALID_CUSTOMER","message":"找不到客戶 99999999，無法建立訂位"}
```

團隊實務備註

- **BOOKID** 自動編號用 `SELECT MAX( bookid ) + 1`，這是教學用的最簡單做法，正式系統通常會用專門的號碼範圍物件（**Number Range Object**，**SNRO** 維護）而不是查表取最大值——`MAX + 1` 在高併發情

境下有 race condition 風險 (見思考題 1) · Number Range Object 由 SAP Kernel 保證原子性遞增 · 才是正式系統該用的做法；這題選擇簡化版本是為了聚焦在「伺服器自動決定新資源身分」這個 REST 概念本身 · 不節外生枝去教 Number Range Object 的維護方式

- **SBOOK** 除了這題用到的 9 個欄位 (`carrid/connid/fldate/bookid/customid/class/passname/forcuram/forcurkey`) 之外還有 `custtype/smoker/luggweight/wunit/invoice/loccuram/loccurkey/order_date` 等必填欄位 (DDIC 定義為 `not null`) · 這題只補了 `order_date = sy-datum` · 其餘留空——這跟 rs06 的 **SFLIGHT** 情境一模一樣 (見 rs06 團隊實務備註) : DDIC 層級的 `not null` 標註不會擋住 ABAP 的 **INSERT** · 只有畫面輸入 (Dynpro/SM30) 層級才會檢查 · 訓練環境這樣處理沒問題 · 正式系統要在程式裡另外補業務規則驗證
- 這題證明了 **rs08** 的分層方式可以重複套用在新資料模型上 : **ZCL_RS09_BOOKING_SERVICE** 從第一行開始就是這樣設計的 (不是先寫在 **Resource** 裡 · 事後才搬) · 這是團隊規範「新功能一開始就分層」而不是「先求有、之後重構」的實務示範——回顧 op12 的教訓：重構永遠比一開始就設計對要花更多力氣

思考題

1. `SELECT MAX( bookid ) FROM sbook WHERE ... + 1` 這個自動編號邏輯 · 如果兩個請求同時對同一個航班建立訂位 (rs06 思考題 3 討論過的 race condition 在這題會更嚴重) · 有沒有可能兩個請求都算出同一個 `lv_next_bookid` · 其中一個 **INSERT** 因為主鍵重複而失敗？這題的 `create_booking` 目前有沒有攔截 **INSERT** 失敗的情況 (回顧 rs06 的 **409 Conflict** 設計) ？如果沒有 · 呼叫端會看到什麼結果？
2. `update_booking` 允許透過 **PUT** 更改 `customid` (把訂位轉給別的客戶) · 並且重新驗證新客戶存在——如果業務需求是「訂位建立後 · 客戶不能更改」 · 這個限制該加在 **ZCL_RS09_BOOKING_SERVICE** 的哪個方法？跟這次「**PUT** 應該整筆覆蓋」的設計原則會不會衝突？
3. 這門課從 rs01 到 rs09 · 一路都是「一個 **Resource** 對應一種資料實體」(航班或訂位) · 如果現在要新增一個「一次查詢某航班的完整資訊 · 包含航班本身跟這個航班底下所有訂位」的 API (也就是 **Header/Detail** 一起回傳) · 這種端點的 URL 該怎麼設計？回傳的 JSON 結構會長什麼樣子 (航班欄位旁邊多一個訂位陣列) ？這跟目前 `/flights/{...}` 和 `/bookings?carrid=...&connid=...` 分開查詢兩次有什麼本質上的差異？

課程總結：REST 課程九題回顧

#	主題	這題新增的能力
rs01	為什麼要 REST	HTTP 動詞語意、三層架構
rs02	Application 與 Resource	路由掛載
rs03	JSON 序列化	<code>/UI2/CL_JSON</code>
rs04	路徑參數	單筆定位
rs05	Query Parameter	動態篩選
rs06	POST 建立	反序列化、外鍵驗證、201+Location
rs07	PUT/DELETE	統一錯誤格式、防禦式 <code>CATCH cx_root</code>
rs08	可測試性	Service/Resource 分層、ABAP Unit

#	主題	這題新增的能力
rs09	期末整合	新資料模型、多重驗證、伺服器自動編號

答案

見 `zcx_rs09_booking_error.clas.abap`、`zcl_rs09_booking_service.clas.abap` (+ `.testclasses.abap`)、`zcl_rs09_app.clas.abap`、`zcl_rs09_bookings.clas.abap`、`zcl_rs09_booking.clas.abap` (SAP 端物件 `ZCX_RS09_BOOKING_ERROR` / `ZCL_RS09_BOOKING_SERVICE` / `ZCL_RS09_APP` / `ZCL_RS09_BOOKINGS` / `ZCL_RS09_BOOKING`)。SICF Service 路徑 `/sap/bc/zrest_training/rs09`。